

Graph Neural Networks for Reliability and Dependability Analysis in Complex Distributed Systems based on Publish–Subscribe Architecture

Abstract

Publish–subscribe middleware decouples producers and consumers, improving scalability but obscuring the dependency chains along which one component’s failure cascades. Runtime telemetry does not exist before deployment, and code-level static analysis is blind to system-level topology, so identifying which components are critical, and why, remains difficult. We present **Software-as-a-Graph (SaG)**, a pre-deployment **Static System Analysis** framework modeling a pub-sub system as a typed, weighted, directed multigraph over five component classes with logical dependencies derived through typed projection rules. On this representation we train a relation-specific **Heterogeneous Graph Transformer** to forecast cascading failure impact, paired with an interpretable Reliability–Maintainability–Availability–Vulnerability (RMAV) attribution baseline, both validated against discrete-event cascade simulators operating on a structurally disjoint view of the same model under an input–label independence guarantee. Across seven synthetic topologies and three graphs transcribed from open-source architectures we report four results. **(1)** Typed learning leads the strongest non-learning baseline on rank correlation ($\rho = 0.730$ vs 0.595), though a paired test does not establish that margin; the critical-set advantage is more robust ($F_1@K = 0.465$ vs 0.308). **(2)** Heterogeneity’s advantage is concentrated in generalisation, not in-distribution fit. **(3)** The two cascade oracles used as ground truth agree only weakly ($\rho = 0.394$), bounding construct validity. **(4)** Edge criticality, measured by simulated removal rather than inferred, shows most individual links replaceable. SaG also runs as a blocking CI/CD quality gate in well under a minute even at 500+ components, though its anti-pattern catalog’s agreement with the cascade oracle is modest and the gate is currently absolute rather than delta-aware.

Keywords: publish–subscribe middleware, architectural dependability, cascading failure, heterogeneous graph neural networks, static system analysis, pre-deployment verification, quality attributes, CI/CD quality gate

1. Introduction

1.1. Motivation

The publish–subscribe (pub-sub) paradigm has become a backbone communication abstraction for large-scale distributed systems, underpinning cyber-physical, cloud-native, robotics, and Internet-of-Things architectures. Its appeal is decoupling: producers and consumers are separated in time, space, and synchronization, so components can be added, removed, or scaled without direct knowledge of one another [1]. Industry standards such as the Data Distribution Service (DDS) and MQTT formalize this model and expose deployment-time choices — topics, brokers, reliability, durability, and other Quality-of-Service (QoS) policies — that materially shape how the system behaves under stress [2, 3].

The same decoupling that makes pub-sub flexible also obscures the dependency structure an engineer must reason about when a component fails. There are no explicit caller–callee edges: an application that publishes to a topic has no static link to the applications that subscribe to it, even though those subscribers are wholly dependent on it for data. Failures do not propagate along a call graph but along *derived* paths — through shared topics and brokers, through colocated deployment nodes, and, distinctively, through shared libraries whose failure strikes every consumer *simultaneously* rather than sequentially. A raw architecture diagram does not reveal these chains, and the components whose failure would be most damaging are frequently not the ones a diagram makes look important.

Crucially, this reasoning is most valuable *before* deployment, when hardening is cheapest, yet pre-deployment is precisely when no runtime telemetry exists to identify weak points empirically. An engineer must therefore answer a hard question from the architecture alone: *which components are critical, and why?* Uncontained cascading failures in production also carry a sustainability cost — emergency re-provisioning, retransmission storms, and failover loops that consume compute and power that preventing the cascade at design time avoids.

1.2. Problem Statement and Limitations of Existing Approaches

We address pre-deployment criticality analysis for pub-sub middleware as two coupled sub-problems. Given only an architectural description — its applications, libraries, topics, brokers, deployment nodes, and QoS policies — we seek to (1) attribute each component an interpretable measure of *how* and *why* it is critical, grounded in the **ISO/IEC 25019:2023 Quality-in-Use** standard, so that a diagnosis directs a specific remediation rather than a generic warning; and (2) predict the cascade impact of each component’s failure, identifying which components should be hardened first. Both must be computed without runtime data and remain explainable: a single opaque criticality number is of limited use to an architect choosing between competing interventions under a fixed budget.

A major “**Architecture-Code Gap**” compounds this: a system can have perfectly clean source code in every component, yet remain highly fragile, because a Single Point of Failure or a mismatched QoS contract in the deployment topology can collapse the entire system on a single crash. Three strands of prior work bear on the gap, and each leaves it open. **Static Code Analysis (SCA)** — SonarQube-style tools evaluating code cleanliness, complexity, and cohesion — is entirely blind to inter-component topology and dynamic middleware cascades. **Runtime dependability and chaos engineering** hardens systems through fault tolerance, replication, and injected failure, but assumes a *running* staging or production system and does not answer what a design should protect before deployment. **Topology-only and homogeneous learning-based centrality** collapses a component’s risk into a single scalar that conflates distinct failure mechanisms (SPOFs vs. cascade hubs), and homogeneous graph neural networks collapse typed pub-sub semantics into a flattened view. No existing approach offers an interpretable, multi-dimensional, pre-deployment attribution over the *typed* pub-sub graph, coupled to code-level SCA metrics, heterogeneous-GNN impact prediction, and automated CI/CD gating. That is the gap this paper fills.

1.3. Our Approach

We present **Software-as-a-Graph (SaG)**, a pre-deployment **Static System Analysis (SSA)** framework. SaG models a pub-sub system as a typed, weighted, directed multigraph over five node types (applications, libraries, topics, brokers, nodes) and derives logical **DEPENDS_ON** dependencies through typed projection rules (Section 3). SaG ingests code-level SCA metrics as vertex attributes and performs multi-dimensional quality attribution, decomposing criticality into orthogonal Reliability, Maintainability, Availability, and Vulnerability (RMAV) dimensions under a stated weighting audited for Analytic Hierarchy Process (AHP) consistency (Section 4) — a baseline whose value we show lies in explanation, not in ranking accuracy (Section 8.3).

SaG then performs **failure-impact analysis**, predicting cascade impact $I(v)$ with two predictors: the interpretable composite $Q(v)$ and a learned **Heterogeneous Graph Transformer (HGL)**. We evaluate the learned predictor in two variants — QoS-masked (HGL) and QoS-encoded ($HGL-QoS$) — to isolate what explicit QoS contract features contribute; Section 8.3 reports that contribution as a null, so every headline figure in this paper is the QoS-masked HGL. Both are validated against a discrete-event simulator under an **input-label independence guarantee** (Section 5). A **prescriptive remediation** stage generates topology-level hardening edits and verifies them on counterfactual graphs in-memory (Section 6), and SaG integrates directly into CI/CD pipelines as a blocking gate that fails the build on CRITICAL or HIGH severity structural anomalies in seconds (Section 6.2) — currently absolute rather than delta-aware, a limitation Section 9.3 scopes as future work.

The paper is organized around four research questions:

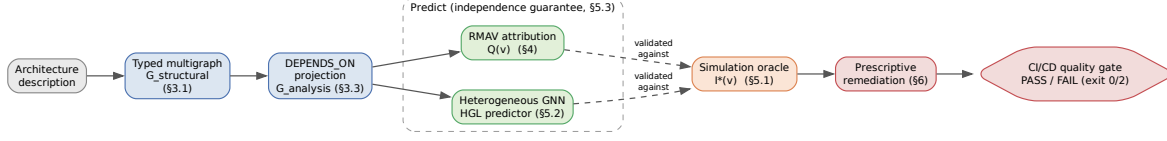


Figure 1: End-to-end SaG pipeline — architecture description → typed multigraph → DEPENDS_ON projection → the two predictor paths and the simulation oracle path, with the independence boundary between them marked → remediation and the CI/CD gate.

RQ1. For pre-deployment criticality prediction, *where* does typed graph learning improve on non-learning structural baselines — in recovering the full impact ordering, in identifying the critical set, or both — and does that answer differ in-distribution versus on unseen architectures?

RQ2. What does taking node and edge type seriously expose that a single-score topological centrality misses, and what does it fail to expose?

RQ3. How does explicit multi-attribute QoS contract feature injection affect in-distribution convergence versus out-of-distribution Leave-One-Scenario-Out (LOSO) generalizability?

RQ4. What is the feasibility and performance overhead of deploying the graph-based analyzer as a blocking Quality Gate in CI/CD pipelines?

RQ5. Does the framework’s predictive ranking transfer to architectures it did not generate?

RQ1 is deliberately phrased as *where* rather than *whether*: the answer depends on which metric is asked about (Section 8.1). RQ1–RQ3 are answered on the synthetic scenario suite (Section 8.1–Section 8.3); RQ4 evaluates gating feasibility (Section 8.4); RQ5 is answered on three real-world graphs (Section 8.5) and carries the paper’s external validity most weakly.

1.4. Contributions

This paper makes the following contributions:

- 1. A typed graph model with hierarchical SCA metric integration.** We define the SaG multigraph and the RMAV decomposition, which propagates code-level quality metrics into global system criticality scores, and train a relation-specific Heterogeneous Graph Transformer over it (Section 3–Section 5).
- 2. A scope condition on where graph learning pays for pub-sub criticality.** Under a single evaluation contract applied to every predictor (Section 7.2), typed learning leads the strongest training-free baseline on both ranking ($\rho = 0.608$ vs 0.521 out of distribution) and critical-set identification ($F_1@K = 0.465$ vs 0.308) — after repairing that baseline, which was silently computing unweighted betweenness on every scenario (Section 8.1). We report the repair because a baseline accidentally identical to the one it should improve on inflates any margin measured against it. The critical-set advantage holds under every protocol and every version of the apparatus; the ranking margin fails a paired significance test in-distribution and rests on an unretained artifact out of distribution (Section 8.1, Section 9.2). The contribution is the scope condition, not the win.
- 3. Multi-dimensional criticality attribution, positioned as explanation rather than accuracy.** RMAV decomposes criticality into four dimensions with distinct remediation owners. A shrinkage sweep shows the dimension weighting does *not* improve ranking accuracy over equal weights (Section 8.3); we scope the contribution to attribution accordingly (Section 4, Section 8.3).
- 4. Measured edge ground truth.** We obtain edge ground truth by simulating removal of each candidate relationship rather than projecting node labels through a heuristic multiplier, finding that most individual links are replaceable and exposing a class of structurally non-redundant edges the cascade model cannot express (Section 8.2).

5. **A prescriptive remediation stage and an automated CI/CD quality gate.** We formalise a Generate→Verify procedure in which every candidate edit is simulated in isolation and admitted only if it improves impact by more than the simulator’s seed noise (Section 6), and a build-blocking gate that evaluates system-level structural risk statically, executing in well under a minute on every scenario in our corpus (Section 6.2, Section 8.4). The gate is absolute rather than delta-aware in the current implementation (Section 9.3).
6. **Empirical real-world validation on open-source software architectures.** We demonstrate SaG’s external validity on three authentic real-world software graphs — Autoware.universe, a Cloud-Native Microservices mesh, and Train-Ticket (Section 7.1, Section 8.5) — achieving mean rank agreement over five seeds ($\rho = 0.688, 0.778, 0.759$) and up to $F_1@K = 1.000$ on two of three, though 5 of 15 gate checks fail across the three graphs and $F_1@K = 1.000$ is partly a tie-breaking artifact (Section 8.5). What the three cases jointly support is that SaG’s predictive ranking generalizes beyond the synthetic generator to independently-sourced architectures across two paradigms, not an unqualified success on production systems.

1.5. Prior Work and Organization

This work extends the authors’ earlier structural baseline — multi-layer graph dependency analysis — introduced in prior work [4]; the present paper consolidates that foundation with the heterogeneous GNN predictor, quality attribution, SCA integration, and CI/CD gating into a single submission targeted at this special issue, with no companion manuscript under review in parallel. The remainder of the paper is organized as follows: Section 2 reviews related work; Section 3 defines the SaG model; Section 4 presents interpretable attribution; Section 5 presents failure-impact analysis; Section 6 covers remediation and CI/CD gating; Section 7 describes the experimental setup; Section 8 reports results (RQ1–RQ5); and Section 9 discusses findings, threats to validity, and conclusions.

2. Related Work

This paper draws on, and contributes to, several established lines of research: publish–subscribe dependability, static analysis techniques, pre-deployment system verification, structural criticality, and multi-criteria quality scoring.

The pub-sub paradigm is a foundational communication abstraction for large-scale distributed systems, valued for decoupling producers and consumers in time, space, and synchronization [1]. Standards such as DDS and MQTT formalize deployment-time QoS choices [2, 3], alongside log-structured brokers such as Kafka [5] and robotics middleware such as ROS 2 [6]. Research on pub-sub dependability has accordingly emphasized runtime fault tolerance, reliable event dissemination, replication, and recovery — approaches that assume observable behavior and react to or mask faults as they occur. Our concern is complementary and earlier in the lifecycle: estimating, from an architectural model alone, which components would have the greatest downstream impact if they failed, before any system is deployed. A large body of runtime hardening work — exemplified by Chaos Engineering [7], popularised by Netflix’s Chaos Monkey — evaluates real operational environments but carries operational risk and occurs late in the lifecycle.

Static verification typically operates at the source-code level: SCA tools such as SonarQube parse source into ASTs to compute complexity [8], duplication, and modular metrics such as LCOM [9, 10], but are blind to inter-component topology. Static System Analysis (SSA) addresses this “Architecture-Code Gap” by modelling the system as a global graph of communicating components, ingesting code-level metrics as node properties and propagating them through the inter-component dependency topology, so that code-level fragility combines with structural fragility to reveal systemic risk. Continuous pre-deployment verification shifts this analysis left into CI/CD pipelines [11, 12]: the architecture is defined as “Architecture-as-Code” via configuration descriptors, and SSA tools run automatically on every pull request, blocking the build when a change introduces critical architectural smells. Mature code-level gates follow the same discipline

— SonarQube’s “Clean as You Code” gate evaluates *new* code against the merge base rather than the accumulated state of the whole codebase, paired with an explicit waiver workflow [13] — and our system-level gate adopts the analogous semantics as future work (Section 9.3), because real architectures legitimately contain intentional, risk-accepted SPOFs that an absolute gate would flag on every build.

Network science offers a mature toolkit for identifying important nodes: degree, closeness and betweenness centrality, articulation points, and PageRank-style scores [14, 15, 16, 17], with studies of node removal [18], cascading failure [19], and interdependent networks [20] deepening the understanding of systemic fragility. Their limitation for our purpose is dimensional collapse — a single centrality score conflates mechanisms (a SPOF, a cascade hub, a maintainability bottleneck) that call for different remedies — and representational collapse: once node and edge types are discarded, a shared library’s *simultaneous* failure mode is indistinguishable from an ordinary edge. A growing body of work instead learns criticality from graph structure — FINDER [21], DrBC [22], and the PowerGraph benchmark [23] — typically building on homogeneous message-passing (GCN [24], GraphSAGE [25], GAT [26]). Pub-sub middleware is intrinsically heterogeneous, and flattening it discards information about how failures propagate; heterogeneous GNNs address this directly (RGCN [27], HAN [28], HGT [29], MAGNN [30]), with over-smoothing in dense, hub-dominated regions a known hazard [31, 32]. Our learned predictor adopts relation-specific message passing for exactly these reasons, but we treat it as one of two predictors rather than the sole contribution, since RQ1 asks *where* such learning improves on non-learning alternatives rather than assuming it always does.

Software quality is conventionally described along attributes such as reliability, maintainability, availability, and security [33]; the Quality-in-Use portion of that model is now its own standard, **ISO/IEC 25019:2023** [34], evaluating stakeholder harm over Beneficialness, Freedom from Risk, and Acceptability. The dependability vocabulary we adopt follows the standard taxonomy [35], and the architecture-evaluation tradition we position against is scenario-based methods such as ATAM [36, 37, 38]. Combining several structural properties into one decision score is a multi-criteria problem, for which AHP offers a pairwise-comparison formalism with an explicit consistency check [39], which we use to *state and audit* weights rather than elicit them (Section 4). A related strand detects architectural anti-patterns and recommends refactorings [40, 41, 42, 43], descending from Martin’s coupling metrics [44] and the technical-debt framing [45, 46]; our prescriptive stage differs in accepting an edit only when a discrete-event simulation confirms the reduction in cascade impact, rather than a static metric improving. In summary, prior approaches address pub-sub dependability at runtime, offer code-level SCA blind to topology, offer structural analysis that conflates failure mechanisms, apply graph learning while discarding typed semantics, or use multi-criteria scoring for prioritization but not as an interpretable attribution over a typed graph. SaG combines a typed multigraph model, multi-dimensional attribution, dual interpretable and learned impact prediction, and a simulation-verified CI/CD quality gate.

3. The Software-as-a-Graph Model

This section defines the graph model on which all subsequent analysis operates: the formal object and its node and edge types (Section 3.1), the derivation of logical dependencies and their QoS-derived weights (Section 3.2), and the two graph views, multi-layer projections, and a running example that thread through the rest of the paper (Section 3.3).

3.1. Nodes, Edges, and the Formal Object

A distributed publish–subscribe system is modeled as a typed, weighted, directed multigraph

$$G = (V, E, \tau_V, \tau_E, w_E, w_V), \quad (1)$$

where the vertex set partitions into five component types, $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}}$, the type functions τ_V and τ_E label vertices and edges, and the weight functions $w_E : E \rightarrow [0, 1]$ and $w_V : V \rightarrow [0, 1]$ encode QoS-derived coupling strength (Section 3.2). The edge set is the disjoint union of *structural* edges

imported directly from the architecture description and *dependency* edges (**DEPENDS_ON**) derived from them (Section 3.2).

Table 1: Node and structural edge types of the SaG model. Each node type carries distinct failure semantics; edges are imported directly from the architecture description.

Node Type	Role	Representative instances
Application	A process that publishes and/or subscribes to topics	ROS 2 node, Kafka producer/consumer, MQTT client
Broker	A message-routing intermediary	RabbitMQ, Mosquitto, DDS middleware
Topic	A named message channel	<code>/sensor/lidar</code> , <code>order.events</code>
Node	A physical or virtual host	server, cloud VM, embedded controller
Library	A shared code dependency	sensor driver, codec, message library
Structural Edge	Direction	Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	component produces messages on the topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	component consumes messages from the topic
ROUTES	Broker $\rightarrow$ Topic	broker routes the topic
RUNS_ON	App/Broker $\rightarrow$ Node	component is hosted on the node
CONNECTS_TO	Node $\rightarrow$ Node	direct network link between hosts
USES	App $\rightarrow$ Library	application depends on the shared library

Retaining these types — rather than collapsing them into a single “communicates-with” relation — is what later lets the framework distinguish failure mechanisms that an untyped graph cannot (Section 3.2, Section 5). Application and Library nodes additionally carry ingested SonarQube-derived code metrics (`cm_*` properties: lines of code, cyclomatic complexity, LCOM, coupling), which feed the Maintainability dimension’s Code Quality Penalty term (Section 4.2) and are how the framework bridges the Architecture–Code Gap at the node level.

3.2. QoS-Aware Weights and Derived Dependencies

Not all dependencies are equally consequential: a **RELIABLE/PERSISTENT** channel carrying critical data couples its endpoints far more tightly than a **BEST_EFFORT/VOLATILE** one. Edge weights $w(e) \in [0, 1]$ are computed from each mediating topic’s declared reliability, durability, and transport-priority QoS values (weighted 0.30/0.40/0.30 — durability outweighs the other two because it governs message-state survival, the precondition for resilience — audited for AHP consistency, Section 4.2) blended 0.85/0.15 with a payload-size term, with a floor of $w(e) = 0.01$ so even zero-QoS components stay visible to attribution. Vertex weights propagate this upward with type-specific aggregation that amplifies a library’s risk by its consumer fan-out, anticipating the blast-radius mechanism below; full formulas are in the replication package.

Structural edges record physical relationships but not *logical* dependency: a subscriber and a publisher on the same topic have no direct structural edge, yet the subscriber wholly depends on the publisher for data. We derive a single semantic relation, **DEPENDS_ON**, always directed from *dependent* to *dependency* (“if the target fails, the source is affected”), through six rules:

Table 2: The six `DEPENDS_ON` projection rules deriving logical dependencies from structural edges.

Rule	dependency_type	Pattern (dependent $\rightarrow$ dependency)	Weight
1	<code>app_to_app</code>	subscriber $\rightarrow$ publisher via a shared topic (incl. transitive <code>USES*1..3</code> chains)	$\max_t w(t)$ over shared topics
2	<code>app_to_broker</code>	publisher/subscriber $\rightarrow$ broker routing its topics	$\max_t w(t)$ over routed topics
3	<code>node_to_node</code>	host $\rightarrow$ host, lifted from Rules 1–2 for colocated apps	lifted $\max w$
4	<code>node_to_broker</code>	host $\rightarrow$ broker, lifted from Rule 2	lifted $\max w$
5	<code>app_to_lib</code>	application $\rightarrow$ library it <code>USES</code> — shared-library blast	$w_V(\text{app})$
6	<code>broker_to_broker</code>	bidirectional, two brokers sharing a host — colocation	$w_V(\text{node})$

When two applications communicate over several shared topics, a single `DEPENDS_ON` edge records the worst-case weight together with a separate coupling count `path_count`, kept out of the weight to preserve the $w \in [0, 1]$ contract — three simultaneous failure vectors between the same pair is structurally more fragile than three independent single-topic links.

Two qualitatively different failure modes. This is the crux of the model. Rule 1 encodes *sequential cascade*: a publisher’s failure starves its subscribers, whose failure may in turn affect their dependents, propagating step by step through topics and brokers. Rule 5 encodes a *simultaneous blast*: when a shared library fails, every application that uses it fails at once, in a single event, not along a propagation path. An untyped graph cannot tell these apart — both look like ordinary edges — yet they demand different predictions and different remedies. Preserving the `app_to_lib` type (Rule 5) is what lets the framework represent this simultaneous-blast mechanism at all, just as preserving `broker_to_broker` (Rule 6) makes broker-colocation risk representable.

3.3. Graph Views and a Running Example

The construction produces two complementary views of the same system, and their separation is load-bearing for the framework’s validity: $G_{\text{structural}}$, the imported structural graph, used by the discrete-event simulators to generate the ground-truth impact labels (Section 5.1); and $G_{\text{analysis}}(\ell)$, the layer-projected `DEPENDS_ON` graph, on which all structural metrics, quality attribution, and prediction are computed (Section 4). Because attribution is computed on G_{analysis} while ground truth is generated by simulating $G_{\text{structural}}$, the predictor’s inputs are kept disjoint from the label-producing path — the **independence guarantee** that makes the pre-deployment claims of Section 4–Section 5 non-circular, and that we state and rely on throughout. G_{analysis} is further filtered into four analytical layers — application, infrastructure, middleware, and system — each isolating a component scope, a dependency subset, and the quality dimension it most informs (Reliability, Availability, Maintainability, and Overall, respectively), and rolled up along a MIL-STD-498 [47] CSU $\rightarrow$ CSC $\rightarrow$ CSCI $\rightarrow$ CSS hierarchy so criticality can be reported at whatever granularity an organization’s configuration management already uses.

Consider three applications a_1, a_2, a_3 , where a_1 publishes to a topic t that a_2 and a_3 subscribe to, all three depending on a shared library ℓ ; a single broker b routes t , and one host n runs all four processes. Derivation adds $a_2, a_3 \rightarrow a_1$ (`app_to_app`, Rule 1), $a_i \rightarrow b$ (`app_to_broker`, Rule 2), $n \rightarrow b$ (`node_to_broker`, Rule 4), and $a_i \rightarrow \ell$ (`app_to_lib`, Rule 5). The two structures encode different risks: losing a_1 degrades a_2 and a_3 through a cascade the simulator propagates over time, whereas losing ℓ fails all three simultaneously. A topology-only centrality score ranks ℓ by ordinary connectivity and cannot represent that its single failure

collapses the whole component group at once — whether that representational difference translates into a scoring gap large enough to matter is an empirical question we test in Section 5.4 (on our synthetic suite, it does not).

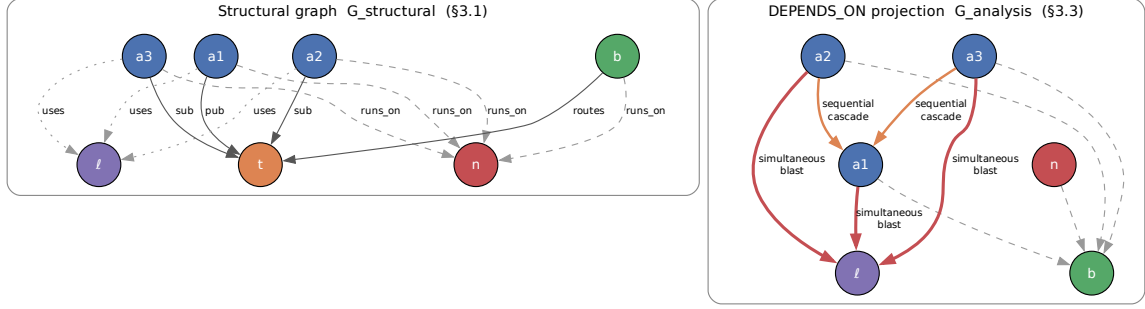


Figure 2: The running example’s structural graph and its derived **DEPENDS_ON** projection, with sequential-cascade and simultaneous-blast edges visually distinguished.

4. Interpretable Attribution as a Baseline

Centrality answers *whether* a component is important with a single number; an architect choosing between a replica, a reroute, and a decoupling refactor needs to know *why*. This section presents the framework’s interpretable diagnostic: a decomposition of each component’s criticality into four orthogonal quality dimensions — Reliability, Maintainability, Availability, and Vulnerability (RMAV) — each computed from disjoint structural metrics and combined into a composite score. Grounded in **ISO/IEC 25019:2023 (Quality-in-Use)** [34], criticality represents the counterfactual loss of beneficialness, freedom from risk, and acceptability that stakeholders would experience if an architectural element failed.

4.1. Four Dimensions and Formal Definitions

Table 3: The four RMAV dimensions, the architectural question each answers, and the engineering role each routes to.

Dim.	Architectural Question	High score means	Remediation owner
R	How broadly and deeply does failure propagate?	Failure cascades widely; hard to contain	Reliability Engineer
M	How hard is this to change safely?	Tightly coupled structural bottleneck	Software Architect
A	Is this a structural single point of failure?	Removing it partitions the dependency graph	DevOps / SRE
V	How attractive a target is this for attack?	Central and reachable on G^T , with many strongly-guaranteed flows converging on it	Security Engineer

For components, the dimensions are **orthogonal by construction**: each raw structural metric feeds exactly one dimension, never more — a deliberate design constraint, not an empirical observation, since

allowing a metric into two dimensions would silently inflate its weight relative to the stated weighting (Section 4.2). Orthogonality is what makes the breakdown legible: a pure single point of failure scores high on A but low on R, M, and V; a god-component scores high on M; a cascade hub scores high on R. The *shape* of the profile names the failure mode.

Two of the framework’s formal definitions do real work in what follows. **Criticality is a consequence, not a risk:** no RMAV dimension estimates how *probable* a component’s failure is, only how much is lost *given* that it does — ranking u above v says that losing u hurts more, not that u is more likely to be lost, and restricting the construct to consequence is precisely what makes it computable pre-deployment, since consequence follows from architecture while likelihood follows from behaviour that does not yet exist. **Criticality is relative, not absolute (D4):** every score and tier is relative to the score distribution of the system being analysed (tiers are box-plot thresholds, Section 4.2) and to the analytical layer, so criticality values are **not comparable across systems or layers** — a well-designed redundant system still has a CRITICAL tier, and a system full of SPOFs still has a MINIMAL one. D4 constrains how this paper may aggregate: any figure computed over more than one scenario must be formed from within-scenario ranks or per-scenario statistics, never from raw scores pooled across systems (Section 5.4, Section 8.5 carry the corresponding scoping statements).

The framework also gives inter-component dependencies the same four-dimensional attribution as components, so that a partial outage — one link down, both endpoints healthy — is scored rather than inferred from endpoint scores. We do not develop this relationship-level construction further here: by the framework’s own independence guarantee (Section 5.3), it cannot currently be validated against the edge-removal measurement of Section 8.2, which operates on a structurally disjoint population (Section 9.3).

4.2. The Composite Score, Classification, and Determinism

All metric inputs are rank-normalized to $[0, 1]$. **Reliability** is driven by Reverse PageRank on the transpose `DEPENDS_ON` graph $G^\top$ (the failure-propagation direction, since edges point dependent $\rightarrow$ dependency), blended with in-degree and a cascade-depth potential amplified by multi-path coupling, with a fan-out form dispatched for Topic nodes, which have no `DEPENDS_ON` in-degree. **Maintainability** is driven by betweenness centrality, QoS-weighted out-degree, and a Code Quality Penalty built from the ingested SonarQube metrics of Section 3.1. **Availability** is driven by the directed articulation score (rather than the undirected version, which both over- and under-reports in pub-sub graphs), amplified by QoS weight so a SPOF carrying critical traffic scores as doubly severe. **Vulnerability** is driven by reverse eigenvector and harmonic closeness centrality on $G^\top$, modelling adversarial reach toward high-guarantee flows. The four combine into

$$Q(v) = w_A A(v) + w_R R(v) + w_M M(v) + w_V V(v) \quad (2)$$

under a stated weighting $(0.43, 0.24, 0.17, 0.16)$ — a design judgement written on Saaty’s 1–9 scale and audited for Analytic Hierarchy Process consistency [39] ($CR \leq 0.10$) rather than elicited from raters — adapted per system toward its aggregate QoS profile, the computable form of D1’s “within its operational context” clause.

We classify with an adaptive box-plot rule applied independently to each dimension and to the composite (CRITICAL: $Q > Q_3 + 1.5 \text{ IQR}$, down to MINIMAL: $Q \leq Q_1$, with a percentile fallback below $n = 12$), so that a component can be CRITICAL on Availability yet MINIMAL on Vulnerability — telling the architect to add a replica rather than harden an interface. Attribution is fully deterministic: every input to $Q(v)$ is a structural metric of G_{analysis} , none derives from the discrete-event simulation that produces the ground-truth labels used to evaluate the framework (Section 5.1) — the **independence guarantee** that makes a measured correlation between $Q(v)$ and simulated impact evidence of genuine predictive content rather than leakage.

We report the sensitivity of this weighting, and it is not favourable. Sweeping a shrinkage parameter that blends the stated weighting toward a uniform prior shows no plateau at any value and a monotone decline in ranking accuracy against simulated impact, with equal weights outperforming the calibrated setting by 0.111 ρ (Section 8.3, Table 11) — the one robustness result in this paper that did not move under re-measurement. We draw the corresponding conclusion about the contribution rather than defending the weighting: **the value of the RMAV decomposition is attribution, not ranking**

accuracy. A composite score ranks; a four-dimensional profile explains *why* a component ranks where it does and routes the finding to the engineering role equipped to act on it — a structural single point of failure and a cascade hub call for different remediations even at identical composite scores — and that explanatory function is unaffected by the weighting result. A practitioner optimising for ranking alone should use equal weights; one who needs to know *why* a component is critical needs the profile, whatever the weights.

5. Failure-Impact Analysis via Heterogeneous GNN and Interpretable Forecasting

Quality attribution (Section 4) tells an architect why a component is structurally critical. This section asks the complementary question: *how much of the system actually fails* when a given component fails, and how well each predictor anticipates it. We define the three simulation oracles that supply ground truth (Section 5.1), the two predictors we evaluate against them (Section 5.2), the independence between predictor inputs and the label path that makes the evaluation sound (Section 5.3), and two checks that take node type seriously (Section 5.4).

5.1. Ground Truth: Three Simulation Oracles

In the absence of runtime telemetry, ground truth is produced by discrete-event failure simulation over the *raw* structural graph $G_{\text{structural}}$ — directly on PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, and USES edges, without the derived DEPENDS_ON projection. For each component v , a failure is injected, the resulting disruption is propagated over a fixed horizon, and the residual service degradation is measured.

The framework contains three such oracles, and they are not interchangeable. We name them here because which one backs a given number materially bounds what that number can support.

- $I^*(v)$ — produced by **FaultInjector**: the mean subscriber feed-loss fraction under a breadth-first cascade. This is the label the learned predictors are trained and evaluated against, and it backs the predictor tables of Section 8.1.
- $I_{\text{comp}}(v)$ — produced by **FailureSimulator**: a four-component weighted composite, $I_{\text{comp}}(v) = 0.35 \text{ reachability_loss} + 0.25 \text{ fragmentation} + 0.25 \text{ throughput_loss} + 0.15 \text{ flow_disruption}$, graded in $[0, 1]$ with weights on the same AHP-audited footing as Section 4.2. I_{comp} backs the validation gates, the checks of Section 5.4, and remediation acceptance (Section 6.1).
- $I_{\text{dyn}}(v)$ — produced by **MessageFlowSimulator**: the drop in delivered message rate that *surviving* consumers experience, obtained not by traversing edges but by discrete-event simulation of actual traffic — each publisher emits at its declared rate, every topic fans out into a bounded per-subscriber queue, and the fault is injected mid-run. I_{dyn} trains nothing and gates nothing: it is reported in Section 7.4 as a construct-validity check on the other two.

I^* and I_{comp} agree only weakly — mean Spearman $\rho = 0.394$ across the seven scenarios (Section 7.4). We therefore treat evidence gathered against one as *not* transferring to a claim measured against the other, and apply that constraint to our own analyses; Section 7.4 quantifies the agreement and label-coverage bounds, and Section 8.2 flags where the distinction bites. The two cascade oracles share a **propagation_threshold** (default 0.2) below which partial feed loss is treated as recoverable degradation rather than a cascade trigger, and — because intra-wave propagation order is tie-broken stochastically — are run over five seeds, with impact reported as the across-seed mean and standard deviation, the latter also the noise scale the remediation filter of Section 6.1 is calibrated against.

5.2. Two Predictors over the Same Model

We evaluate two predictors of simulated cascade impact, deliberately spanning the interpretability–capacity spectrum. The **interpretable predictor** is the composite $Q(v)$ of Section 4, computed deterministically on G_{analysis} with no learned parameters. The **learned predictor** is a **Heterogeneous Graph Transformer (HGT)** [29] that assigns relation-specific attention and message-passing parameters across the five node types (App, Broker, Topic, Node, Library) and the structural/DEPENDS_ON edge types, so that message transformations differ by the semantic relation they traverse rather than being shared across a flattened graph: each relation gets its own projection matrices inside the attention computation, and the learned attention weight on an edge is directly interpretable as how much that specific typed relation contributed to a neighbour’s representation.

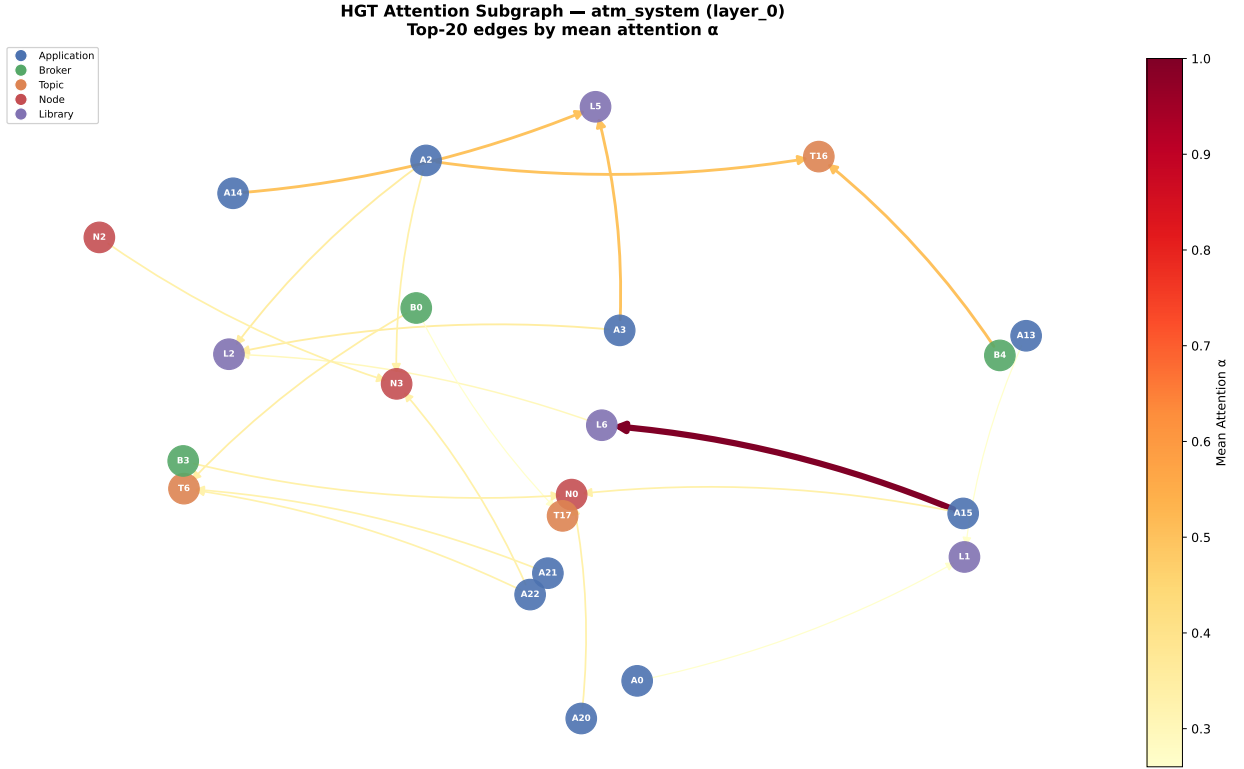


Figure 3: Learned relation-specific attention over a case-study subgraph, with per-edge α from the HGT layer’s own softmax.

The *HGL-QoS* variant additionally injects the continuous QoS attributes into the edge-attention aggregation, scaling message magnitude by interface contract strength; the base **HGL** variant masks these fields to isolate the contribution of typing alone from that of QoS encoding (RQ3, Section 8.3). Both variants consume features from G_{analysis} (not the simulator) and are trained inductively against the $I^*(v)$ labels of Section 5.1. For RQ1 we report the two predictors separately rather than blended, so that *where* typed learning improves on the interpretable score is settled on like-for-like rankings.

5.3. The Independence Guarantee

The evaluation is only meaningful if the predictor cannot see its own labels. Two structural properties enforce this: predictors operate on G_{analysis} while both simulators operate on $G_{\text{structural}}$, so label-producing and feature-producing computations are distinct passes over distinct graph views; and no simulation output is ever fed back as an input feature to $Q(v)$ or the learned predictor. For the learned predictor specifically, this is also checked at inference time: the GNN service raises before a forward pass if a feature tensor carries

a target label attribute, a defensive check against a leakage bug introduced later in this codebase’s lifetime rather than a proof none exists today. The remediation stage (Section 6) obeys the same discipline: its candidate-generation phase never reads simulated impact.

5.4. Two Checks That Take Node Type Seriously

Two analyses probe whether the typed model’s distinctions matter empirically, both measured against $I_{\text{comp}}(v)$ and both reported as negative or consistency results rather than as headline findings.

The shared-library blast mechanism. Rule 5 (Section 3.2) gives libraries a structurally distinctive *simultaneous* failure mode, motivating the hypothesis that a library’s composite score $Q(v)$ would understate its true cascade impact. Tested directly across all seven scenarios (165 Library nodes), we did **not** find the hypothesized mismatch: the highest composite score reached by any library is $Q = 0.422$, and across every library in the corpus $I_{\text{comp}}(v)$ never exceeds $Q(v)$ — the composite score is, if anything, mildly conservative for this node type. We report this as a negative result rather than omit it: the mechanism itself remains a real structural distinction worth preserving in the model, and we retain the FanOutReduction operator’s blast-radius trigger (Section 6.1) as a structurally motivated safeguard rather than as a mechanism validated by this particular result (Section 8.2 restates this alongside the measured edge-criticality finding it is adjacent to but does not corroborate).

Stratified correlation. A single pooled correlation between $Q(v)$ and simulated impact could in principle mask a Simpson’s-paradox-style effect if node types occupy sufficiently different regions of the (Q, I) plane. Pooling across all seven scenarios (1,545 nodes) gives $\rho = 0.374$; computed separately by node type, correlations range from 0.322 (Topic) to 0.429 (Broker), all significant at $p < 0.01$. We do **not** find a Simpson’s-paradox effect: the pooled figure sits inside the per-type range rather than diverging from it, which is a useful consistency result — the predictive relationship holds at similar strength across every component type — even though the check happened not to overturn the pooled conclusion here. The effect it looked for *does* occur elsewhere in this study: pooling Application and Library nodes into a single correlation moved HGL on `av_system` from $\rho = 0.836$ within Applications to 0.46 pooled (Section 8.2), a case where a pooled figure was actively misleading and one that went unnoticed until the evaluation contract of Section 7.2 was imposed. Per D4 (Section 4.1), this pooled figure is a diagnostic over the union of seven within-system rankings, not a cross-system criticality result, and no claim elsewhere in the paper rests on it.

6. Prescriptive Remediation and CI/CD Quality Gating

Attribution (Section 4) and impact analysis (Section 5) are diagnostic: they tell an architect *which* components to harden and *why*. This section closes the loop with a prescriptive stage that proposes concrete architectural edits and verifies that they reduce simulated failure impact before any deployment, and describes how the diagnostics are operationalised as a continuous CI/CD quality gate.

6.1. Generate–Verify and the Remediation Operators

Remediation runs in two strictly separated phases, preserving the independence discipline of the rest of the framework. **Generate:** given G_{analysis} and its attribution, four operators propose candidate topology edits from structure alone, never reading simulated impact:

Table 4: The four remediation operators, their structural triggers, and the failure mode each targets.

Operator	Structural trigger	Edit applied	Failure mode targeted
RedundancyInsertion	directed articulation point / high A SPOF	add a redundant instance or redistribute responsibilities	graph-partitioning SPOF

Operator	Structural trigger	Edit applied	Failure mode targeted
PathDiversification	bridge edge / single routing path for a topic	add an alternative route (e.g. a second routing broker or network link)	fragmentation on a non-redundant edge
FanOutReduction	high structural blast radius (topic subscriber fan-out; library consumer count)	interpose an intermediary or split the over-shared channel	simultaneous blast / fan-out explosion
SharedTopicReduction	high multi-path coupling (large <code>path_count</code> / MPCII between a pair)	decouple redundant shared topics between the pair	multi-channel coupling fragility

Verify: each candidate edit is applied to produce a counterfactual graph, on which the `FailureSimulator` (Section 5.1) is re-run from scratch; the edit is accepted only if

$$\Delta I > \kappa \sigma_{\text{seed}} \quad (3)$$

— the reduction in I_{comp} exceeds $\kappa = 1.0$ times the simulator’s own across-seed noise — at *every* sampled propagation threshold, evaluated per candidate edit before it is committed to the policy. This separation matters: a stage that both proposed and scored edits using the same signal would be optimizing against itself, and normalising the acceptance bar by σ_{seed} ties it to the fragility of the cascade at that point rather than to an arbitrary constant. `FanOutReduction`’s trigger is deliberately *not* the composite $Q(v)$ but a direct structural blast-radius signal (subscriber fan-out, consumer count), so that a low- Q , high-blast component is still selected — the remediation-side expression of the paper’s central claim that single-score criticality is insufficient, independent of whether Section 5.4 finds such a component in this particular suite. No `Verify` result feeds back into `Generate` within a run, so there is no closed-loop search that would reintroduce the circularity the framework is built to avoid.

Running this procedure across the scenario suite ($\kappa = 1.0$, three propagation thresholds, three seeds — reduced from five because acceptance requires a full re-simulation per candidate per threshold, the most expensive experiment in the study), 162 of 332 candidate edits (48.8%) clear the filter, with no scenario in the suite regressing under the per-edit filter, removing by construction the failure mode of an earlier, unverified design in which a regressing edit could be carried by an improving one. Acceptance is decided on singletons, not subsets: nothing in the procedure establishes that a set of individually-accepted edits remains beneficial applied together. The acceptance rate varies widely with topology — 3 of 35 candidates on `Autonomous Vehicle` against 38 of 58 on `IoT Smart City` — concentrated where a fan-out bottleneck actually exists, and the resulting System Risk Index reductions are real but small across the suite (+0.0025 to +0.0158), which is the result we report rather than a demonstration that the prescriptive stage is yet practically valuable (Section 9.3).

6.2. The CI/CD Quality Gate

To operationalise these diagnostics, SaG integrates into developer workflows as a blocking Quality Gate: when a pull request introduces Architecture-as-Code changes, the pipeline executes `detect_antipatterns.py`, which runs the full anti-pattern catalog against the candidate topology and issues an exit code — 0 (no anomalies, deployment permitted), 1 (medium-severity smells, permitted with warnings), or 2 (CRITICAL/HIGH severity anomalies — SPOFs, cyclic dependencies, broker overload — build broken, **deployment blocked**). The underlying machinery is in-memory via the thread-safe `MemoryRepository` port and does not require a live database connection, which is what makes the runtimes of Section 8.4 possible.

This is an *absolute* gate: every run evaluates the candidate topology’s full finding set, not a diff against a prior baseline. This has a known consequence we do not paper over: a real architecture carrying an intentional, risk-accepted single point of failure — a sole-source feed, a deliberately unreplicated legacy broker — fails the build on every commit, indistinguishable from a genuine regression. A *delta-aware* gate

that evaluates the candidate against a merge-base topology, blocking only on newly introduced findings, together with a waiver register recording accepted risk so it stays visible rather than silently re-triggering, would close this gap; we describe the design in Section 9.3 as future work rather than claim it here, since the mechanism is not implemented in the released tool.

7. Experimental Setup

This section describes the data, predictors, metrics, and protocols used to answer RQ1–RQ4 (Section 8). The design follows one overriding principle, carried from the independence guarantee (Section 5.3): every predictor is evaluated against the same simulator-derived ground truth produced by an independent process, so the claims we make are *comparative* — which modeling choices perform better under identical conditions — rather than assertions of absolute accuracy in operational deployments.

7.1. Datasets

Synthetic suite. We evaluate on seven synthetic pub-sub scenarios spanning distinct deployment domains — autonomous vehicles, high-frequency trading, clinical healthcare integration, centralized hub-and-spoke enterprise systems, distributed IoT smart-city telemetry, cloud-native microservices, and large-scale enterprise pub-sub — produced by a statistical topology generator and ranging from 50 to 300 applications, exercising fan-out-dominated, dense-pub-sub, and anti-pattern/SPOF regimes.

Real-world open-source suite. To test operational generalizability on authentic software graphs, we evaluate SaG on three real-world architectures: **Autoware.universe** (ROS 2 autonomous driving [48]), a cyber-physical pub-sub architecture with 32 Applications, 24 Topics carrying explicit DDS QoS profiles, 3 Brokers, 6 Deployment Nodes, and 10 shared C++ libraries; a **Production Cloud-Native Microservices Mesh** based on the Google Online Boutique benchmark [49], 22 microservices, 20 Topics across Kafka/RabbitMQ/Redis PubSub/NATS, 6 Kubernetes nodes, and 8 shared libraries; and **Train-Ticket**, the Fudan University railway-booking benchmark [50], 41 microservices, 30 Topics, 3 Brokers, 8 deployment nodes, and 8 shared libraries — at 90 components, the largest of the three.

Pooled across all synthetic and real-world scenarios, the evaluation corpus exercises 1,770 components. The synthetic suite is registered in a manifest carrying a canonical SHA-256 per dataset, and a regression test regenerates each from its configuration and fails on any divergence, so the corpus is byte-reproducible from its configs; the three real-world graphs are hand-transcribed by a dedicated adapter rather than the statistical generator, so the byte-identity guarantee applies to the synthetic suite only. All seven synthetic scenarios are used for the predictor evaluation (Section 8.1–Section 8.3), the checks of Section 5.4, and the remediation evaluation of Section 6.1.

7.2. Predictors, Baselines, and Evaluation Metrics

Table 5: Predictors and baselines, and the factor each contrast isolates.

Predictor	Description	Role
RMAV / Q	deterministic multi-dimensional composite (Section 4)	interpretable predictor
HGL	heterogeneous graph transformer, QoS-masked	learned predictor (typed)
HGL-QoS	heterogeneous graph transformer, QoS-encoded	learned predictor (typed + QoS)
GL / GL-QoS	homogeneous GAT on the type-collapsed projection	learning baseline (untyped)

Predictor	Description	Role
Topo-BL / Topo-QoS	structural centrality (betweenness, articulation points; QoS-weighted)	non-learning baseline

The contrast **Topo-*** vs learned isolates the value of learning (RQ1); **GL** vs **HGL** isolates the value of typed heterogeneity; **HGL** vs **HGL-QoS** isolates explicit QoS encoding (RQ3); and **RMAV/Q** vs the learned predictors isolates when interpretable attribution suffices. Structural baseline features are kept decoupled from GNN inputs so no comparison leaks information across the predictor boundary.

We report metrics in three families. **Ranking**: Spearman ρ against $I^*(v)$ is primary, complemented by NDCG@10 and Top-5/Top-10 overlap for the practically relevant case where only a few components can be hardened. **Identification**: precision/recall/F1 for critical-component detection, plus SPOF-F1 for articulation-point classification against simulated availability impact. **Statistical rigor**: bootstrap 95% confidence intervals ($B = 2000$) [51, 52] and paired Wilcoxon signed-rank tests [53] ($p < 0.05$) for predictor comparisons. Following Section 5.4’s stratified check, ρ is always reported *by node type* in addition to any pooled figure. The implementation also carries a topology-class-adjusted validation gate (default targets $\rho \geq 0.70$, $F1 \geq 0.80$, tightened or relaxed per class; Section 8.5 reports gate failures against these).

One evaluation contract, one sample. Every variant in every table is scored by the same function on the same node set — a correction rather than a description of prior practice: an earlier version of this study scored predictor families on different populations, and correcting it raised every learned variant by 0.35–0.48 ρ in-distribution while leaving the baselines essentially unchanged (Section 9.2). The evaluation key set is a function of the graph and labels only, never of any variant’s predictions; the reported figure is held-out, from one train/validation/test split pinned by node identity and shared across variants (a full-population score would flatter a trained model by including nodes it was fitted on); and a variant that cannot cover the declared population fails loudly rather than silently shrinking its sample. **Absent is not zero**: a stratum whose predictions or labels are constant has an *undefined* rank correlation and is reported as such, never as 0.0 — Topic and physical Node components carry no simulated ground truth at all (Section 7.4), and reporting them as 0.0 would present a labelling gap as a measured model failure.

7.3. Protocols

Two evaluation regimes answer different generalization questions. **In-distribution (per-scenario)**: predictors are computed and compared against that scenario’s own simulated ground truth — the regime for RQ1 and RQ2, asking how well attribution and learned predictors recover the criticality ordering of a *known* system. **Inductive (Leave-One-Scenario-Out)**: to test generalization to *unseen* architectures — the true pre-deployment condition — LOSO cross-validation trains the model on six scenarios (with the largest used for early stopping) and evaluates on the seventh, whose nodes never participate in any forward pass and whose labels never enter any loss. Every configuration is run over five seeds {42, 123, 456, 789, 2024}; reported scores are seed means, and the across-seed standard deviation is both reported and reused as the noise scale in the remediation acceptance criterion (Section 6.1). The remediation sweep of Section 6.1 is the one exception, using the first three seeds for the compute reason stated there.

7.4. Ground Truth: Three Oracles and Their Agreement

The three oracles of Section 5.1 are constructed differently, measure different quantities, and are not interchangeable; conflating them is the most likely way to over-read a result in this paper.

Table 6: The three simulation oracles, what each measures, and which results rest on which.

Symbol	Engine	Quantity	Used for
$I^*(v)$	FaultInjector	Mean subscriber feed-loss fraction under a BFS cascade	Learned-predictor labels; Tables 8 and 10 (Section 8.1); the sensitivity sweeps of Section 8.3
$I_{\text{comp}}(v)$	FailureSimulator	0.35 reachability + 0.25 fragmentation + 0.25 throughput + 0.15 flow	Validation gates; the checks of Section 5.4; remediation acceptance (Section 6.1)
$I_{\text{dyn}}(v)$	MessageFlowSimulator	Delivery-rate loss suffered by <i>surviving</i> consumers, by discrete-event simulation of traffic	Reported construct-validity check only — no labels, no gates, no tables

Measured agreement between the two cascade oracles is weak. Across the seven scenarios, mean Spearman $\rho = 0.394$ and mean top-20% Jaccard = 0.286, ranging from $\rho = 0.578$ (Enterprise) down to $\rho = 0.092$ (Hub-and-Spoke, effectively uncorrelated). All seven correlations are positive — a weak convergent-validity argument — but at $\rho \approx 0.39$ it is weak, and we apply the resulting constraint to our own analyses: a result established against one oracle is not evidence for a claim measured against the other (Section 8.2 flags where this bites). **I_{dyn} agrees with I^* far more strongly:** mean $\rho(I_{\text{dyn}}, I^*) = 0.765$, minimum 0.548 at Hub-and-Spoke — where I^* and I_{comp} collapse to near-independence. Because I_{dyn} reaches this ranking by simulating traffic through queues rather than traversing `DEPENDS_ON`, this rules out the cascade *algorithm* as the source of I^* ’s ranking (Section 9.2), though top- K membership is the weaker half — mean top-20% Jaccard 0.316 — so this corroborates *ranking*, not critical-set identification.

Label coverage and the noise ceiling. The cascade model has no rule expressing the failure of a Topic or a physical Node, so those types carry no ground truth at all — 30–47% of components per scenario are unlabelled and excluded from scoring rather than scored as zero — and Broker labels are degenerate in three of seven scenarios. The three oracles do not cover the same components, so every agreement figure above is computed over the intersection. And the labels have a reproducibility ceiling: across seeds, the ground truth agrees with *itself* at test–retest ρ of 0.807–1.000 and top-20% Jaccard 0.44–1.00 (post an instrument-defect fix disclosed in Section 9.2). No method can exceed the former, and every top- K metric inherits the latter.

7.5. Model Configuration and Implementation

The learned predictors are implemented in PyTorch Geometric [54]. Table 7 fixes every hyperparameter, identical across all learned variants so the contrasts of Section 7.2 isolate architecture and features rather than tuning budget; values were fixed before the reported runs and not tuned per scenario, since a per-scenario search would leak held-out information under the in-distribution protocol.

Table 7: Learned-predictor configuration. Identical across HGL, *HGL-QoS*, GL and GL-QoS except where the architecture differs by construction.

Component	Setting
Convolution	HGTConv (heterogeneous); GATConv for the homogeneous GL variants
Layers	3
Hidden channels	64
Attention heads	4
Dropout	0.2
Input projection	per-node-type linear $\rightarrow$ LayerNorm $\rightarrow$ ReLU

Component	Setting
Output heads	four RMAV residual MLPs + one composite head, sigmoid-activated
Optimizer	AdamW, learning rate 3×10^{-4} , weight decay 1×10^{-4}
LR schedule	CosineAnnealingWarmRestarts , $T_0 = \max(50, \text{epochs}/4)$, $T_{\text{mult}} = 2$, $\eta_{\min} = 0.01 \cdot \text{lr}$
Gradient clipping	max-norm 1.0
Epochs / early stopping	300, patience 30 on validation loss
Node splits	60% train / 20% validation / 20% test, pinned by node identity (Section 7.2)
Loss	composite MSE + 0.5 multitask + 0.3 ListMLE ranking + 0.1 pairwise margin + 0.1 RMAV consistency

Hardware and runtime. Training and evaluation were run on a single workstation; the LOSO sweep is the dominant cost, at roughly 31 minutes for HGL and 36 for *HGL-QoS* across all folds and seeds, against 5–6 minutes for the homogeneous variants and well under a minute for the training-free baselines. The CI/CD gate measurements of Section 8.4 were taken on the same machine rather than on hosted runner hardware, so they should be read as an order-of-magnitude feasibility result rather than as a calibrated figure for any particular CI provider.

8. Results

8.1. RQ1 — Interpretable Attribution versus Learning

Every figure in this section is produced by one evaluation contract (Section 7.2): all six variants are scored on an identical held-out node set, drawn from a single train/validation/test split pinned by node identity and shared across variants — the previously reported version of this table did not have that property, and the correction changes its conclusion (Section 7.2).

In-distribution, typed learning leads on the point estimate. Table 8 reports Spearman ρ against $I^*(v)$ on the held-out split, averaged over five seeds, with bootstrap 95% confidence intervals and the held-out sample size n :

Table 8: In-distribution held-out Spearman ρ against $I^*(v)$, seed means over $\{42, 123, 456, 789, 2024\}$ with bootstrap 95% CIs. n is the number of held-out Application-type components scored in that scenario (Section 7.2).

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	<i>HGL-QoS</i>
AV System	16	0.308 [0.13, 0.46]	0.750 [0.55, 0.91]	0.760 [0.66, 0.84]	0.655 [0.40, 0.88]	0.713 [0.51, 0.88]	0.692 [0.47, 0.87]
Enterprise	60	0.393 [0.29, 0.50]	0.797 [0.75, 0.85]	0.853 [0.81, 0.89]	0.513 [0.21, 0.79]	0.885 [0.86, 0.91]	0.883 [0.84, 0.92]
Financial Trading	12	0.246 [-0.05, 0.56]	0.709 [0.58, 0.84]	0.851 [0.82, 0.89]	0.874 [0.85, 0.89]	0.882 [0.85, 0.91]	0.903 [0.88, 0.92]
Healthcare	10	-0.182 [-0.40, 0.06]	0.772 [0.59, 0.88]	0.815 [0.76, 0.87]	0.804 [0.74, 0.85]	0.842 [0.80, 0.87]	0.845 [0.80, 0.88]
Hub-and-Spoke	14	0.299 [0.12, 0.48]	0.511 [0.21, 0.76]	0.494 [0.21, 0.72]	0.475 [0.34, 0.63]	0.537 [0.43, 0.65]	0.557 [0.47, 0.65]
IoT Smart City	40	-0.063 [-0.17, 0.05]	0.068 [-0.07, 0.20]	0.674 [0.54, 0.81]	0.474 [0.27, 0.67]	0.891 [0.87, 0.91]	0.883 [0.86, 0.91]

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	<i>HGL-QoS</i>
Microservices	18	0.302 [0.07, 0.54]	0.556 [0.38, 0.71]	0.524 [0.41, 0.63]	0.436 [0.31, 0.56]	0.362 [0.05, 0.55]	0.354 [0.18, 0.53]
Mean	—	0.186 [0.02, 0.32]	0.595 [0.41, 0.74]	0.710 [0.60, 0.81]	0.604 [0.49, 0.73]	0.730 [0.58, 0.86]	0.731 [0.57, 0.86]

The heterogeneous predictor leads the strongest non-learning baseline by $\Delta\rho = +0.135$ (HGL 0.730 vs Topo-QoS 0.595). Its lead over the *homogeneous* learned baseline is much narrower — +0.020 over GL — and not uniform: GL wins on AV and Microservices, HGL wins decisively on IoT (0.891 vs 0.674). In-distribution, therefore, these data do **not** establish that relation-specific message passing is what supplies the learned margin; the two learned families are separated by less than the across-seed spread. The claim that typing matters is carried by the out-of-distribution and in-domain k-fold results below, not by this table.

Significance testing, and what it does and does not license. Table 9 reports the paired Wilcoxon signed-rank test across the seven scenarios:

Table 9: Paired Wilcoxon signed-rank tests across the seven scenarios ($n = 7$; two-sided).

Comparison	$\Delta\rho$	Scenarios won	W	p	
HGL vs Topo-BL	+0.544	7/7	0.0	0.016	significant
HGL vs GL-QoS	+0.126	6/7	5.0	0.156	n.s.
HGL vs Topo-QoS	+0.135	5/7	8.0	0.375	n.s.
GL vs Topo-QoS	+0.116	5/7	5.0	0.156	n.s.
HGL vs GL	+0.020	5/7	11.0	0.688	n.s.
<i>HGL-QoS</i> vs HGL	+0.001	3/7	13.0	0.938	n.s.

Only the comparison against the unweighted structural baseline reaches significance. **The in-distribution margin over Topo-QoS — the +0.135 quoted above — is not established by a paired test across scenarios** ($p = 0.375$), and neither is the +0.020 over the homogeneous model. Two readings are needed together: the test is genuinely underpowered (at $n = 7$ paired scenarios, the smallest attainable two-sided p is 0.016, so only a clean 7-of-7 sweep can ever reach $p < 0.05$, and “not significant” is not evidence of no effect) — but it is equally not evidence of an effect. We therefore state the in-distribution ranking result as a point-estimate lead this design cannot confirm, and do not treat it as the paper’s load-bearing evidence for learning; that role belongs to the critical-set result below and, with the qualification stated there, to the transfer protocols. ($Q(v)$ does not appear in Table 8: the in-distribution harness scores it under a separate path that does not emit held-out per-scenario correlations, and we leave the cell empty rather than quote a figure computed under a different contract; Section 8.3 characterises its in-distribution ranking behaviour directly, and its inductive result is in Table 10.) Two boundary conditions frame the table: the ground truth agrees with *itself* at test–retest ρ of 0.807–1.000 (Section 7.4), so HGL’s mean 0.730 sits within the spread of what it is scored against; and Microservices — by construction the sparse, low-centralisation topology — is where every learned predictor is weakest (HGL 0.362), while the structural baselines degenerate on Healthcare and IoT ($\rho \leq 0$). No predictor in this study is uniformly best.

Out of distribution, the typed model leads. Under Leave-One-Scenario-Out — the true pre-deployment condition, in which the model must rank a system whose cascade dynamics it has never seen:

Table 10: Inductive Leave-One-Scenario-Out evaluation. Cross-fold mean ρ against $I^*(v)$, across-fold standard deviation, and $F_1@K$ on the held-out scenario.

Variant	Mean ρ (LOSO)	Std ρ	$F_1@K$	Training required
Topo-BL	0.105	0.151	0.179	no
Topo-QoS	0.521	0.305	0.308	no
RMAV / $Q(v)$	-0.093	0.140	0.209	no
GL (homogeneous)	0.436	0.120	0.440	yes
GL-QoS (homogeneous)	0.430	0.125	0.435	yes
HGL (typed)	0.608	0.177	0.465	yes
<i>HGL-QoS</i> (typed + QoS)	0.595	0.190	0.461	yes

Topo-QoS requires no training, no labels, and no transfer assumption, so its out-of-distribution score is its score; HGL reaches $\rho = 0.608$ against its 0.521, and leads on $F_1@K$ too (0.465 vs 0.308). The same ordering holds under in-domain k-fold, wider still — HGL-QoS 0.693 and HGL 0.666 against Topo-QoS 0.492 — where the typed models are also the most stable across folds ($\sigma = 0.07$ against 0.34).

This comparison is only meaningful because the baseline was repaired first. In an earlier revision, Topo-QoS scored *no* QoS weighting at all: the lookup for $w(t)$ targeted the wrong graph element, every derived dependency edge kept a unit weight, and the QoS-weighted baseline silently computed plain betweenness — it was Topo-BL under another name. We resolve $w(t)$ from the shared Topic instead. A baseline accidentally identical to the one it is meant to improve on will always flatter whatever it is compared against, and the figures above are reported only after that defect was removed (Section 9.2).

Unlike Table 8, the run behind Table 10 was not persisted to a retained artifact. Table 8 regenerates exactly from a stored result file; the corresponding LOSO result file was overwritten during the revision, and the most recent retained run log — produced *before* the baseline repair — records a different ordering, with Topo-QoS at $\rho = 0.609$ against HGL at 0.597, a tie. We report the post-repair figures because they come from the corrected apparatus, but three changes are confounded in the interval between the two runs — the baseline repair, the corpus regeneration, and the ground-truth cache rebuild — and we cannot attribute the shift to the repair alone. Readers should treat Table 10’s *ranking* margin as provisional pending a re-run under the final apparatus with the artifact retained, and weight the $F_1@K$ column, which favours the typed model under both runs, accordingly (Section 9.2).

The learned advantage is most robust in set identification. $F_1@K$ favours the typed model by a wider relative margin than ρ does — a 51% relative improvement — with the same ordering holding for both homogeneous learned baselines over both structural ones, and unlike the ρ comparison this does not invert under any of the three protocols we ran.

RQ1 therefore resolves as a **scope condition rather than a verdict**. Learning leads on the point estimate under all three protocols, on both metrics — but the *ranking* half is not established: it fails the paired significance test in-distribution ($p = 0.375$), it rests on an unretained artifact out of distribution, and the strongest non-learning comparator is degraded on 2 of 7 scenarios. The set-identification advantage is the more defensible half. Three caveats bound even that: the LOSO across-fold standard deviation remains substantial (0.177 for HGL), top- K metrics inherit the label churn of Section 7.4 (the ground truth’s own top- K set agrees with itself at Jaccard 0.44–1.00), and $Q(v)$ does not transfer at all as a *ranker* ($\rho = -0.093$ LOSO) — a negative result about its ranking use, not about its attribution use (Section 8.3). The practical recommendation we defend is correspondingly narrow: use typed learning when the deliverable is a shortlist, and treat the ranking comparison as open.

8.2. RQ2 — What Taking Node and Edge Type Seriously Shows (and Does Not Show)

We report three analyses that take node and edge *type* seriously: one positive result and two measured/negative ones (the shared-library and stratified checks are reported in Section 5.4; this section restates only their bearing on the current comparison, not their full figures).

Heterogeneity is the dominant source of predictive gain — but only where the model must generalise. Isolating architecture from QoS encoding, the typed model’s advantage over the homogeneous baseline is negligible in-distribution ($\Delta\rho = +0.020$) and grows sharply moving away from the training distribution: $+0.172$ under LOSO (0.608 vs 0.436) and $+0.257$ under in-domain k-fold (0.666 vs 0.409); the typed model is also far more stable across folds ($\sigma = 0.07$ vs 0.15 for GL). When train and test come from the same topology, a homogeneous GAT can recover most of what typing provides, because the type signal is largely redundant with structure it can observe directly; when the model must rank a system whose cascade dynamics it has never seen, relation-specific message passing is what carries over. On $F_1@K$ the two learned families are much closer out of distribution (0.465 vs 0.440), so this is a claim about ranking transfer specifically.

Edge criticality, now measured rather than inferred. Earlier versions of this framework labelled edges by projecting node labels through a hand-chosen bridge multiplier — an assumption about edge importance, not an observation of it. We now remove each candidate relationship — leaving both endpoints active, the partial-outage case that distinguishes edge from node criticality (Section 4.1) — and recompute impact against a no-op control, since the impact function is non-zero on an untouched graph (topics already lacking a publisher or subscriber count as lost throughput). The candidate set on `av_system` — structural bridges united with the highest-betweenness structural edges — contains 50 edges (35 `RUNS_ON`, 11 `CONNECTS_TO`, 3 `SUBSCRIBES_TO`, 1 `PUBLISHES_TO`), of which exactly 4 carry non-zero impact: the one `PUBLISHES_TO` edge and all three `SUBSCRIBES_TO` edges, each connecting a shared library to the topic it produces or consumes. Two findings follow. First, **most individual links are replaceable**, in both magnitude and count: the largest measured impact of severing any single relationship is 0.00504, over an order of magnitude below the largest single-*component* impact in the suite ($I_{\text{comp}} = 0.320$), and 46 of 50 candidates measure exactly zero. Second, the measurement exposes a modelling gap the heuristic concealed: every one of the 46 `RUNS_ON/CONNECTS_TO` candidates — structurally non-redundant bridges by construction — scores exactly zero, because the cascade routes no traffic over infrastructure-layer relations at all; the measurement means *this model cannot express that link’s failure, not that link does not matter*.

A methodological note. The candidate set above requires the sweep to run against a freshly loaded repository, before any structural analysis has touched it: running the analysis and prediction stages against the same in-memory repository instance *before* constructing the simulator’s graph view causes derived edges to leak into what the simulator receives as $G_{\text{structural}}$ — a repository state-ordering issue distinct from the import-level independence check of Section 5.3, and not visible in this paper’s other results since the standard pipeline order is Simulate before Predict throughout, but a reproducibility hazard for this specific measurement.

Scoping caveat. The library and stratified checks of Section 5.4, and this edge-measurement finding, are computed against $I_{\text{comp}}(v)$, whereas Tables 8 and 10 are computed against $I^*(v)$; the two oracles agree at mean $\rho = 0.394$ (Section 7.4), so neither transfers to the I^* -backed tables, and we flag this rather than let adjacency in the text imply mutual support.

8.3. RQ3 and Robustness — Ablations and Sensitivity

QoS encoding (RQ3): a null result in all three regimes. Adding explicit QoS edge attributes to the typed model moves accuracy by less than the across-seed spread, and the *sign of the effect depends on the protocol*: in-distribution $\rho = 0.731$ for *HGL-QoS* against 0.730 for HGL ($+0.001$), out of distribution 0.595 against 0.608 (-0.013), and under in-domain k-fold 0.693 against 0.666 ($+0.027$). An effect that changes direction across evaluation protocols while remaining an order of magnitude smaller than the fold-to-fold variance is a null. The plausible reading is that the lifted dependency topology already encodes most QoS-relevant routing, so the extra dimensions mainly enlarge the parameter space.

Dimension-weight sensitivity: no plateau, and equal weights win. Sweeping the shrinkage parameter λ that blends the stated dimension weighting toward a uniform prior ($\lambda = 0$ equal weights, $\lambda = 1$ the raw judgement), with the QoS-profile adaptation of Section 4.2 active throughout as in every run reported in this paper:

Table 11: AHP shrinkage sensitivity. Mean ρ against $I^*(v)$ as λ blends the stated weighting toward a uniform prior.

λ	0.00	0.50	0.60	0.65	0.70	0.75	0.80	0.90	1.00
mean ρ	0.292	0.206	0.191	0.187	0.181	0.174	0.167	0.152	0.140

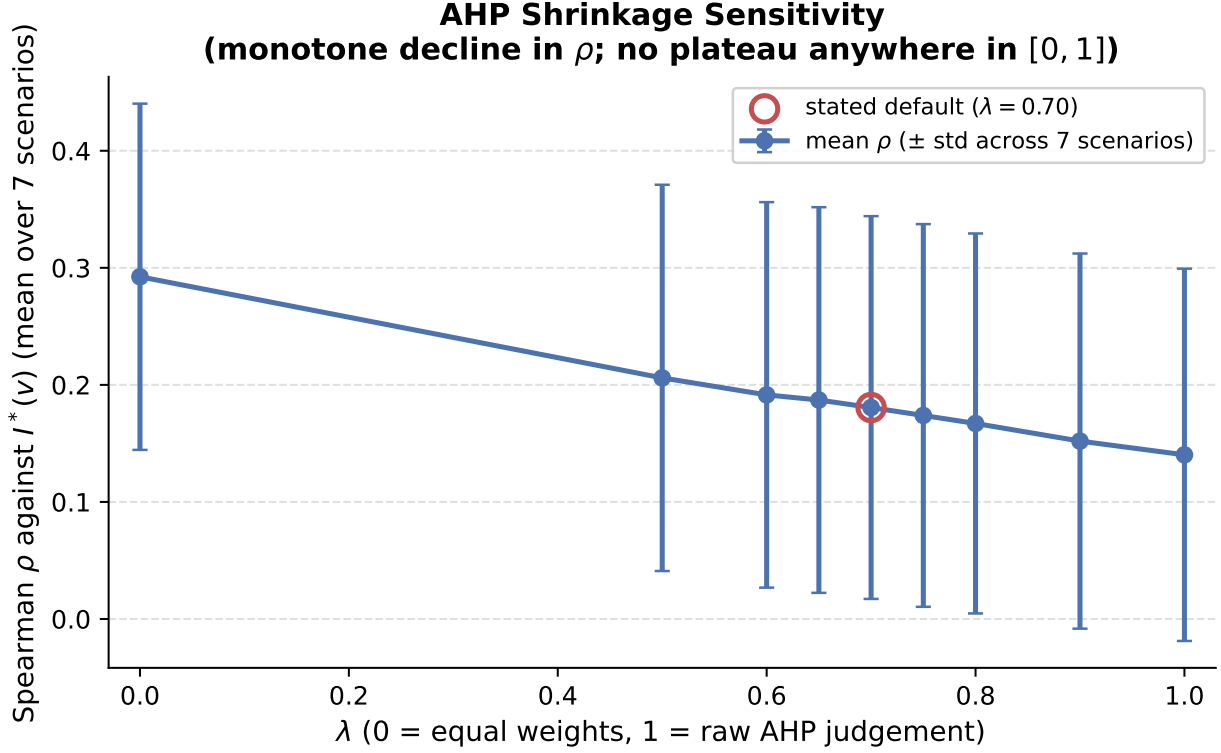


Figure 4: Mean ρ against λ , showing the monotone decline and the absence of a plateau.

ρ is monotonically decreasing in λ ; equal dimension weights outperform the calibrated $\lambda = 0.70$ setting by 0.111. The value of the RMAV decomposition is attribution, not ranking accuracy (Section 4.2); the sweep removes any claim that the specific weights improve predictive accuracy, and a practitioner optimising for ranking alone should use equal weights.

Normalisation sensitivity. The default rank-based normalisation discards magnitude before the weighted sum, making $Q(v)$ closer to a Borda count than a weighted aggregate. Measured against I^* : rank (robust) $\rho = 0.181$, min-max 0.318, z-score 0.318 — retaining magnitude is worth $\approx +0.137\rho$. The outlier-robustness argument for rank normalisation is real but outweighed here; we retain the default so previously reported figures remain interpretable, and report the sweep alongside.

Propagation-threshold sensitivity. Because the ground truth itself depends on `propagation_threshold`, we report ρ across its range:

Table 12: Propagation-threshold sensitivity. Mean ρ against $I^*(v)$ across the sweep; the canonical default is 0.20.

threshold	0.00	0.10	0.20	0.35	0.50	0.75	1.00
mean ρ	0.001	0.109	0.194	0.227	0.226	0.230	0.231

The conclusions *do* depend on this parameter: ρ spans 0.230 across the sweep, the canonical 0.2 default sits below the plateau the curve reaches from 0.35 upward, and at 0.0 — where any feed loss triggers a cascade — the correlation vanishes entirely. We therefore do not claim threshold-independence; remediation edits (Section 6.1) are required to improve impact across the entire sweep precisely because a single-threshold result is not trustworthy here.

8.4. RQ4 — Feasibility and Performance of SaG as a CI/CD Quality Gate

A primary blocker for continuous Static System Analysis is execution time: developers will bypass or disable quality gates that introduce significant build delays. Measuring the wall-clock cost of the structural analysis and anti-pattern catalog via the in-memory `MemoryRepository`, across all eleven scenarios (mean over five seeds): ≤ 90 components (`tiny_system` and the three real-world graphs) cost 0.02–0.04 s; the remaining six generated scenarios (98–326 components) cost 0.27–1.24 s, not scaling monotonically with component count (`hub_and_spoke_system`, 139 components, costs more than `iot_smart_city_system` at 326, consistent with cost being driven by specific detectors’ complexity rather than raw count); and `enterprise_system`, the largest at 520 components, costs 26.74 ± 0.32 s. All eleven complete well under the several-minute budget continuous build pipelines typically allow.

Gate efficacy is harder to state directly, since the gate is currently absolute rather than delta-aware (Section 6.2), so there is no merge-base diff to evaluate detection against; what we can and do measure is the anti-pattern catalog’s raw agreement with the cascade oracle. Scoring CRITICAL/HIGH findings against the oracle’s own critical set gives, across five seeds: precision 0.237 ± 0.014 , recall 0.887 ± 0.059 , Cohen’s $\kappa = -0.036 \pm 0.049$ on the seven generated scenarios, and precision 0.402 ± 0.059 , recall 0.861 ± 0.105 , $\kappa = 0.296 \pm 0.100$ on the three real-world architectures. Recall is high and precision is not: the catalog over-flags relative to the oracle’s critical set, and near-zero κ on the generated corpus means the agreement it does show is close to what chance flagging at the catalog’s own base rate would produce. We read this as a genuine limitation of the pattern catalog as a *stand-alone* predictor of simulated impact — a pattern catalog and a rank predictor serve different purposes — but the gap is wide enough that the catalog’s findings should not be read as impact-calibrated; the composite $Q(v)$ of Section 4 remains the ranking signal validated against the oracle in Section 8.1.

On the remediation side (Section 6.1), 162 of 332 candidate edits (48.8%) clear the multi-threshold acceptance filter across the full seven-scenario suite, with yield concentrated where a fan-out bottleneck exists (3 of 35 candidates on Autonomous Vehicle against 38 of 58 on IoT Smart City) and System Risk Index improvements real but small (+0.0025 to +0.0158 across the suite).

From a sustainability standpoint, evaluating architectural risk statically in-memory (0.02–26.74 s across our corpus) plausibly yields energy savings relative to spinning up staging clusters or running heavier dynamic checks per build, though we have not measured that comparison directly.

8.5. RQ5 — Real-World Open-Source System Architecture Validation

To evaluate operational generalizability beyond synthetic topology generation, we evaluate SaG on the three real-world architectures of Section 7.1. All three rows are produced by the framework’s validation sweep over five seeds, with no QoS enrichment, against the component-level cascade oracle of Section 5.1: Spearman ρ and Kendall τ are computed over the Application population (the other four node types carry constant or near-constant simulated impact on these graphs, per the same coverage limitation as the synthetic suite, Section 7.4); K is $\lceil 0.20 \times |V| \rceil$ within that population (15 of 32 for Autoware, 12 of 22 for Cloud Microservices, 18 of 41 for Train-Ticket); and all three are classified **sparse** by the tool’s topology-class

rule. Reported ρ is the seed mean $\pm$ standard deviation: **FaultInjector** tie-breaks intra-wave propagation stochastically, so — unlike the deterministic RMAV/ $Q(v)$ scores — the simulated labels genuinely vary across seeds within one sweep, though the *ranking* is stable (Rank Consistency Rate = 1.000 for all three).

Table 13: Real-world open-source architecture validation, five seeds, against the component-level cascade oracle of Section 5.1. Architectures are named in full in Section 7.1: Autoware.universe (ROS 2), the Cloud-Native Microservices Mesh, and the Train-Ticket Railway Booking Mesh.

Architecture	Nodes	Apps	ρ (mean $\pm$ std)	τ	$F_1@K$	Tie-rob. $F_1@K$	Non-zero I	Pred. Gain	SPOF-F1	Gate
Autoware	75	32	0.688 $\pm$ 0.009	0.517	0.800	0.800	19/32	+0.360	0.500	FAIL
Cloud Microservices	60	22	0.778 $\pm$ 0.001	0.639	1.000	0.760	8/22	+0.014	0.333	FAIL
Train-Ticket	90	41	0.759 $\pm$ 0.001	0.605	1.000	0.810	14/41	+0.264	0.571	FAIL

Rank correlation is strong on two of three architectures and closest to the framework’s own gate on Train-Ticket, but all three fail it overall. Cloud Microservices ($\rho = 0.778 \pm 0.001$) and Train-Ticket ($\rho = 0.759 \pm 0.001$) clear the $\rho \geq 0.75$ gate threshold; Autoware does not (0.688 ± 0.009 , short by 0.062) and carries the most seed-to-seed variance of the three. All three nonetheless fail the **sparse-topology** gate as a whole, on SPOF-F1 ≥ 0.6 : Autoware 0.500, Cloud Microservices 0.333, Train-Ticket 0.571 — closest of the three, short by only 0.029. Cloud Microservices additionally fails predictive gain ≥ 0.02 (0.014). We report the failing gate rather than the passing correlations alone, because presenting one without the other would overstate what these three cases establish.

Set-containment of the genuinely critical components is real; the reported $F_1@K = 1.000$ on two of three graphs is partly a tie-breaking artifact, and we report both. On Cloud Microservices and Train-Ticket, K exceeds the number of Applications carrying non-zero simulated impact, so the “actual top- K ” set is padded with components tied at $I = 0$, and because both predicted and actual orderings use the same stable sort, the tie-padding lands on the same arbitrary components in both — which drives $F_1@K$ to a perfect 1.000. Re-sorting under 200 random shuffles of the tied region gives a tie-robust $F_1@K$ of 0.760 (Cloud Microservices) and 0.810 (Train-Ticket); Autoware is unaffected (19 non-zero exceeds $K = 15$, no boundary tie, $F_1@K = 0.800$ under both measures). The genuine, tie-independent finding is **set containment**: every non-zero-impact Application in Cloud Microservices and Train-Ticket falls somewhere inside the respective predicted top- K . On Autoware, 12 of the predicted top-15 fall in the actual top-15, correctly including the perception/localization hubs; the three misses include `vehicle_cmd_gate`, a false positive where $Q(v)$ ranks it 6th by structural score but its simulated impact is zero on this topology. We checked whether the same padding could inflate the synthetic-suite $F_1@K$ figures of Section 8.1 and found it does not (the smallest margin between K and non-zero-impact count across the seven scenarios is never reached there) — this is a real-world instance of the general caveat Section 7.4 states: a structural score can be confidently wrong about a component the cascade model does not route traffic through in a way that registers impact.

Predictive gain over degree centrality is real but small and graph-dependent, failing its own threshold on one of three graphs. SaG’s $|\rho|$ exceeds degree centrality’s by +0.361 on Autoware, +0.264 on Train-Ticket, and +0.014 on Cloud Microservices — the last below the 0.02 gate threshold, meaning typed dependency semantics add essentially nothing over raw degree on that particular graph.

Four scoping conditions apply, and they matter because this is the paper’s only evidence outside the generator. These are hand-built models of real architectures, not harvested artifacts: what transfers is the *topology and QoS structure* of a real system, not its runtime behaviour. The ground truth is still simulated — no incident record, operator judgement, or observed failure enters, and Section 9.2’s construct-validity bound applies unchanged. At 75, 60, and 90 components these graphs are smaller than five of the seven synthetic scenarios, and because criticality is relative to a system’s own distribution (D4, Section 4.1), the three ρ values are separate within-system results, not points on a shared scale. And they are three systems from two paradigms — Cloud Microservices and Train-Ticket are both microservice meshes, so only Autoware represents a distinct cyber-physical paradigm. What the three cases do establish is the narrower claim that the framework runs end-to-end on externally-specified architectures and recovers a ranking at least as well as on generated ones, on two distinct meshes independently — evidence against the

concern that performance depends on regularities of our own generator, without settling it.

9. Discussion, Threats to Validity, and Conclusion

9.1. Interpretation

The results converge on a single message: for pre-deployment criticality analysis of pub-sub middleware, *how* a component is critical is at least as important as *whether* it is — and the case for graph learning is narrower, and more specific, than we expected when we set out.

Learning pays, most defensibly for set identification (RQ1). The typed predictor leads the strongest non-learning baseline on both metrics under all three evaluation protocols. Out of distribution — the genuine pre-deployment condition — it reaches $\rho = 0.608$ against 0.521 for a QoS-weighted centrality that requires no training, and $F_1@K = 0.465$ against 0.308. We place substantially more weight on the second metric, for three reasons: operationally, an architect hardens a handful of components, not a ranked list of 150; empirically, *Topo-QoS* carries the largest across-fold variance of any predictor in the study; and evidentially, the ranking margin does not survive a paired test in-distribution ($p = 0.375$) and rests, out of distribution, on a run whose artifact was not retained (Section 8.1). If a team needs the critical set, typed learning is worth its training cost on the evidence here; if they need a cheap ordering, QoS-weighted centrality remains a serviceable default, and we cannot presently demonstrate that learning beats it.

Decomposition is worth having for reasons that are not accuracy (RQ2, RQ3). The dimension weighting does not improve ranking, and the stratified check we ran to detect Simpson’s-paradox masking did not find it in the $Q-I$ relation. What survives is the property that actually motivates the decomposition: a four-dimensional profile says *why* a component is critical and routes the finding to an owner, which a scalar cannot, regardless of how the four are combined. The methodological discipline was not wasted either: pooled-versus-stratified reporting caught a real distortion elsewhere in this study, where collapsing Application and Library nodes into one correlation moved a headline figure by 0.38 (Section 5.4).

Edge criticality benefits from being measured rather than assumed. Replacing a hand-chosen bridge multiplier with actual edge-removal simulation reversed the intuition it encoded: most individual links turn out to be replaceable, and a whole class of structurally non-redundant edges (*RUNS_ON*) carries no measurable impact at all because the cascade model cannot express their failure. A plausible label-generating assumption is not a substitute for the observation it stands in for.

Remediation is now verified per edit, which is a stronger guarantee than the previous aggregate but not yet a demonstration of value. Each candidate is simulated in isolation and admitted only if it beats the simulator’s own noise at every propagation threshold. What that filter reveals is that operator yield is highly topology-dependent and the resulting risk reductions are uniformly small. We regard this as the correct outcome of an honest test: the previous design reported a more favourable aggregate precisely because it never asked each edit to justify itself.

Automated quality gating operationalises these checks continuously (RQ4). By evaluating in-memory and bypassing database round-trips, the framework runs anti-pattern scans and counterfactual simulations in seconds, viable as a blocking CI/CD check that bridges the Architecture-Code Gap at commit time — currently an absolute gate rather than a delta-aware one (Section 6.2, Section 9.3), a scope we state plainly rather than claim the sustainable, waiver-aware semantics we have not yet implemented. Of the four contributions this is the one least disturbed by the audit, and the one we would defend most confidently.

9.2. Threats to Validity

Construct validity. D1 and D2 (Section 4.1) define criticality as Quality-in-Use loss, and this study never observes Quality-in-Use directly. The validation chain has two links: ① structural/learned score $\rightarrow$ simulated failure impact, which Section 8 measures and reports as a real, falsifiable result; and ② simulated impact $\rightarrow$ real Quality-in-Use loss, which is not measured anywhere in this paper — no user study, expert elicitation, or production incident record is used, and the simulator is itself a *model* of stakeholder harm

rather than an observation of it. The defensible claim is therefore: *RMAV and the learned predictors track simulated failure impact, and simulated failure impact is our stated operationalisation of Quality-in-Use loss*. The stronger claim — that these scores track Quality-in-Use as stakeholders would report it — is not supported by anything here. Two qualifications keep this from being either overstated or unduly bleak: Effectiveness and Efficiency are in principle measurable from quantities the simulators already produce (delivery rate before/after a fault, latency shift), so link ② is partly closable by re-summarising existing output rather than by new instrumentation; but Acceptability and Satisfaction are behavioural and not measurable by these means at all, which bounds what this construct can ever claim on them. None of that has been *run*; it establishes measurability, not a measurement.

Because the ground truth is simulated rather than observed, the strongest claims we can make remain comparative. Four further bounds apply. *The two oracles agree weakly*: $I^*(v)$ and $I_{\text{comp}}(v)$ correlate at mean $\rho = 0.394$ (Section 7.4), so results established against one do not transfer to claims measured against the other — Section 5.4’s checks are I_{comp} results and are not evidence about the I^* -backed tables of Section 8.1.

Six silent instrument defects were found and corrected during this revision, and are recorded here because each had, or could have had, a published figure resting on it: the Topo-QoS baseline computing no QoS weighting on all seven scenarios (repaired; Tables 8 and 10 recomputed); HGT attention extraction silently capturing nothing because the pinned PyTorch Geometric release exposes no `return_attention_weights` argument (repaired; Figure 3 now uses real per-edge attention); `FaultInjector`’s cascade iterating an unordered Python set salted per-process by `PYTHONHASHSEED`, making labels reproducible only within one interpreter run (repaired; verified identical $I^*(v)$ across five `PYTHONHASHSEED` values); a training-target lookup keying by an attribute the underlying dataclass does not have, silently zeroing the RMAV-consistency loss term for any checkpoint trained outside the harnesses that produced this paper’s tables (repaired); a parallel remediation worker ignoring the configured layer and checkpoint (repaired; the reported Table 4 run is unaffected, since it used exactly the unpatched default); and a post-loop impact computation reading a stale intermediate variable, a no-op under the `cascade_depth_limit=0` setting every reported figure in this paper uses (repaired). Only the third defect changes reported figures — it retroactively bears on two: the label test–retest ceiling of Section 7.4, and Section 8.5’s Autoware row, whose earlier-reported “sweep-to-sweep instability” was this defect rather than a property of that graph. The common thread is that all six were silent — none raised an exception or produced an obviously wrong number — and full detail on each, including which specific tables were checked unaffected and how, is retained in the replication package’s validation log.

A third of each system is unlabelled. The cascade model cannot express the failure of a Topic or a physical Node, leaving 30–47% of components per scenario without ground truth; predictions for them are produced but never validated, and the per-type results report those strata as undefined rather than zero. *Reported figures approach the labels’ own reproducibility*: the ground truth agrees with itself at test–retest ρ of 0.807–1.000 and top- K Jaccard of 0.44–1.00 across seeds; a model scoring near the former has saturated the labels rather than underperformed, and every top- K metric inherits the latter’s churn. *The behavioural oracle is delivery-based, not QoS-aware*: I_{dyn} ’s discrete-event engine resolves topic QoS from an attribute key the generated corpus does not write, so every run falls back to defaults, and I_{dyn} should be read as a *throughput* oracle rather than a QoS-conformance one.

Internal validity. The chief internal risk is circular validation — a predictor scoring well because its inputs leaked from its labels. The framework addresses this by *view* separation (Section 3.3, Section 5.3): predictors operate on G_{analysis} while ground truth is generated by simulating $G_{\text{structural}}$, and no simulation output is fed back as a predictor feature. **This is view independence, not independence of data source**: both views are deterministic functions of the same input topology, so what is ruled out is feature–label feedback, not the possibility that both encode a shared modelling assumption — a distinction that matters for how much weight the guarantee can bear. I_{dyn} narrows this specific charge: because it reaches its ranking by simulating traffic through queues rather than traversing `DEPENDS_ON` and still recovers I^* ’s ordering (Section 7.4), the cascade *algorithm* is not the artifact, though all three oracles remain deterministic functions of the same generated topology, and a modelling assumption shared by the architecture model itself would be invisible to every one of them.

Two further internal-validity issues surfaced during a pre-submission audit and are disclosed because they invalidated previously reported numbers: the evaluation harness scored different predictor families

on different node populations (Section 7.2’s correction changed the sign of the RQ1 conclusion), and the Leave-One-Scenario-Out sweep reused stale model checkpoints and was therefore not training at all — the same command produced $\rho = -0.576$ in 3.2 s against a dirty workspace and $\rho = +0.594$ in 322 s against a clean one. Both are fixed, but the episode is itself a finding: a silently-cached artifact is indistinguishable from a trained one in the output, and only the implausible wall-clock time exposed it. **Artifact retention is uneven across the reported tables, and one headline table cannot currently be regenerated:** Table 8 and the sensitivity sweeps of Section 8.3 regenerate exactly from stored result files, but the LOSO result file behind Table 10 does not exist — it was overwritten during the revision, and the most recent retained log predates the baseline repair and records a different ordering (Section 8.1). No figure should enter a manuscript unless the artifact that produced it is retained and the figure can be recomputed from it; Table 10 is the outstanding exception, and re-running it under the final apparatus is the first item of remaining work.

External validity. This is the weakest dimension of the study, and we regard it as the highest-value follow-up (Section 9.3). The corpus spans ten deployment domains, but only three architectures are not ours: seven scenarios come from a single statistical topology generator, and the three real-world graphs achieve mean rank correlation over five seeds of $\rho = 0.688, 0.778$ and 0.759 . None of the three clears the framework’s own gate — all three fail SPOF-F1 — and the $F_1@K = 1.000$ figures are partly a tie-breaking artifact; we read the result as evidence that predictive *ranking* transfers beyond the generator, not as a demonstration of production readiness. The paradigm count is two, not three: Train-Ticket and Cloud Microservices are both microservice meshes, so cyber-physical pub-sub is represented by Autoware alone, and Leave-One-Scenario-Out evaluation confirms inductive transfer only across held-out *synthetic* architectures, since all seven share a generator.

Conclusion validity. Criticality scores and simulated impact metrics exhibit heavy-tailed, non-parametric distributions that violate normality assumptions. To prevent classification bias, we apply non-parametric rank correlations (Spearman ρ), top- K Jaccard metrics, and adaptive box-plot thresholding ($Q3 + 1.5 \text{ IQR}$) rather than parametric z-scores or arbitrary absolute cutoffs (Section 4.2).

9.3. Limitations and Future Work

Several limitations point to concrete next steps, ordered here by how much they would change the paper’s claims.

Real-world deployment validation and HIL execution. Section 8.5 narrows the external-validity gap on three real-world architectures but does not close it: the graphs are hand-transcribed, their ground truth is still simulated, and none clears the framework’s own validation gate in full. Validating against runtime hardware-in-the-loop fault injection, and against harvested rather than transcribed architectures, remains the highest-value follow-up.

Out-of-distribution ranking is not yet a solved problem. Section 8.1 shows a training-free QoS-weighted centrality matches the learned models on LOSO rank correlation, with the learned advantage confined to critical-set identification. Whether that ceiling reflects the difficulty of cross-architecture transfer, the label noise floor (Section 7.4), or a limitation of the architecture is not resolved here; distinguishing those explanations, plausibly by training on a larger and more diverse scenario corpus, would determine whether typed learning has more to offer than it currently demonstrates.

The dimension weighting does not improve accuracy. We have repositioned RMAV as an attribution mechanism accordingly (Section 4.2, Section 8.3), but a weighting *derived* rather than asserted — fitted to simulated impact, or elicited from a practitioner panel with reported inter-rater agreement — would let the decomposition make an accuracy claim as well as an explanatory one.

Remediation is verified but not yet demonstrably effective. The per-edit acceptance filter removes the possibility of an unverified regressing edit being applied, but the risk reductions it achieves are real and small, and yield depends heavily on whether the topology contains a fan-out bottleneck. Expanding the operator set, deriving the acceptance multiplier κ from broader variance data, and verifying subsets rather than singletons — the current design admits only edits simulated alone — are needed before the prescriptive stage can claim practical value or compositional safety.

The CI/CD gate is absolute, not yet delta-aware. Section 6.2 implements exit-code gating on a candidate topology’s full finding set without diffing against a merge-base topology, so an architecture carrying an intentional, previously-accepted risk fails the build on every commit rather than only when a change introduces something new. A delta-aware gate together with an auditable waiver register is the fix; neither is implemented today, and closing this gap is engineering work rather than an open research question. A related, smaller gap is that `detect_antipatterns.py` requires a live database connection even though the underlying machinery is already usable through the database-free `MemoryRepository`.

Relationship-level attribution is defined but not validated, and closing that gap is out of scope for this submission (Section 4.1): it is scored on `DEPENDS_ON` edges while the removal oracle of Section 8.2 severs raw structural edges, and the two are never defined on a common population. The one route that would close the gap without violating the independence guarantee — tracking, for each derived edge, which raw edges mediate it and aggregating their measured impact — requires a real modelling decision (the mediating relations are many-to-many) rather than a mechanical lift, and we leave it as future work.

Finally, the endpoint for all of the above is calibration against observed failure data from instrumented deployments, which would convert this paper’s comparative claims into absolute ones.

9.4. Conclusion

We presented Software-as-a-Graph, a pre-deployment Static System Analysis framework that models distributed pub-sub middleware as a typed, weighted, directed multigraph and analyzes it along two coupled axes: an interpretable RMAV attribution baseline that decomposes each component’s criticality into orthogonal dimensions, and failure-impact analysis, which predicts cascade impact with both that composite and a learned heterogeneous graph transformer, validated against discrete-event simulation under a strict input-label independence guarantee. A prescriptive remediation stage generates topology-level hardening edits from structure alone and verifies each individually against the canonical simulator, admitting it only when its benefit exceeds the simulator’s own seed noise at every propagation threshold; under that filter 162 of 332 candidates survive across the seven scenarios, but the resulting risk reductions are small and concentrated in the two topologies with pronounced fan-out structure, which we report as the substantive — and qualified — result of that stage.

Integrated directly into pipelines as a blocking CI/CD Quality Gate — currently absolute rather than delta-aware — the framework verifies architectural changes and blocks regression in seconds, bridging the Architecture–Code Gap at commit time. Across a synthetic scenario suite, the framework establishes a scope condition on where typed graph learning pays: it leads a training-free QoS-weighted centrality on both ranking ($\rho = 0.608$ vs 0.521 out of distribution) and identifying which components belong on a shortlist ($F_1@K = 0.465$ vs 0.308), both measured only after repairing that baseline. Alongside that, measuring edge criticality by removal rather than inferring it from endpoints shows most individual links to be replaceable and exposes a class of relations the cascade model cannot express at all, and stratified rather than pooled reporting caught a distortion that moved a headline figure by 0.38. By taking the *type* of every component and dependency seriously, the framework recovers structure that untyped, single-dimensional methods discard, and does so at the point in the lifecycle where it is most valuable: before the system runs.

CRediT authorship contribution statement

[Omitted for double-anonymised review. To be completed on acceptance with the standard CRediT roles: Conceptualization; Methodology; Software; Validation; Formal analysis; Investigation; Data curation; Writing – original draft; Writing – review and editing; Visualization; Supervision.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Omitted for double-anonymised review.]

Data availability

The seven synthetic scenario datasets, their generator configurations, and the manifest of canonical dataset hashes are included in the replication package, from which every synthetic dataset regenerates byte-identically. The three real-world architecture graphs of Section 7.1 and their adapter are included on the same terms. Result artifacts are provided for the in-distribution evaluation (Table 8), the sensitivity sweeps of Section 8.3, the edge-removal measurement of Section 8.2, the remediation sweep of Section 6.1, and the three real-world validation runs of Section 8.5. **One exception is recorded rather than glossed:** the per-fold artifact behind the Leave-One-Scenario-Out results of Table 10 was not retained and cannot be regenerated from the archive without re-running the sweep; Section 8.1 and Section 9.2 state what follows from that. A link to the archived package will be supplied on acceptance.

Declaration of generative AI use

[To be completed by the authors in accordance with the journal’s policy.]

References

- [1] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [2] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [3] OASIS, MQTT version 5.0, OASIS Standard (2019).
- [4] [Anon-A], Authors’ prior work on multi-layer graph dependency analysis for publish–subscribe systems, citation withheld for double-anonymised review (n.d.).
- [5] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [7] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [8] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering* SE-2 (4) (1976) 308–320.
- [9] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [10] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [11] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [12] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.

- [13] SonarSource, Clean as you code, SonarQube documentation (2024).
- [14] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [15] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [16] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [17] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [18] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [19] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [20] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [21] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [22] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [23] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [24] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [25] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [26] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [27] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [28] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [29] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [30] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [31] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: *Proc. AAAI Conference on Artificial Intelligence*, 2018, pp. 3538–3545.

- [32] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, X. Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: Proc. AAAI Conference on Artificial Intelligence, 2020, pp. 3438–3445.
- [33] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [34] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [35] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.
- [36] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.
- [37] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.
- [38] S. Newman, Building Microservices: Designing Fine-Grained Systems, O’Reilly Media, 2015.
- [39] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [40] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [41] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.
- [42] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
- [43] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.
- [44] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
- [45] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.
- [46] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
- [47] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
- [48] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.
- [49] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).

- [50] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [51] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [52] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
- [53] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [54] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.